



AWS Lambda — Complete Notes

Serverless functions · Run code without managing servers

A reader-friendly, all-in-one reference for AWS Lambda.

1. ⚡ What is Lambda

AWS Lambda runs your code **in response to events** without you provisioning or managing servers. You upload a **function**, Lambda runs it on demand, scales automatically, and you pay only for the **compute time consumed** (per millisecond). No idle servers, no patching the OS.

💡 **Mental model:** Lambda is **Function as a Service (FaaS)**. You own the function code; AWS owns everything below it — servers, scaling, availability, OS.

Common uses: API backends (with API Gateway), event processing (S3/DynamoDB/Kafka), stream processing, scheduled jobs (cron), glue between services, real-time file/data transforms, automation.

2. 🧩 Core Concepts

Concept	Description
Function	Your code + config (runtime, memory, role, handler).
Handler	The entry-point method Lambda invokes (<code>file.method</code>).
Event	The JSON input passed to the handler.
Context	Runtime info (request id, remaining time, log group).
Runtime	Language environment: Node.js, Python, Java, Go, .NET, Ruby, or custom.
Trigger / Event Source	What invokes the function (S3, API GW, EventBridge...).
Execution Role	IAM role giving the function permissions to call AWS.
Concurrency	Number of simultaneous executions.

3. 🔄 The Execution Model

1. An **event** arrives from a trigger.
2. Lambda provisions an **execution environment** (micro-VM / Firecracker) if none is warm.
3. It runs **init** code (outside the handler) once per environment.
4. It invokes the **handler** with the event + context.
5. The environment is **frozen** and **reused** for later invocations (warm), then eventually torn down.

Invocation types:

Type	How	Example
Synchronous	Caller waits for the response	API Gateway, ALB, direct invoke
Asynchronous	Event queued; Lambda retries on failure	S3, SNS, EventBridge
Stream / poll-based	Lambda polls and batches records	Kinesis, DynamoDB Streams, SQS, Kafka

💡 Put expensive setup (DB connections, SDK clients) in the **init** phase (module scope) so warm invocations reuse it.

4. 🔔 Triggers & Event Sources

Lambda integrates with 200+ services. Common triggers:

- **API Gateway / Function URL / ALB** — HTTP APIs and webhooks.
- **S3** — object created/deleted (process uploads).
- **EventBridge / CloudWatch Events** — schedules (cron) and event rules.
- **SNS / SQS** — pub-sub and queue processing.
- **DynamoDB Streams / Kinesis** — stream processing.
- **MSK / self-managed Kafka** — message consumption.
- **Cognito, Step Functions, IoT, CloudFront (Lambda@Edge)** — and many more.

5. ⚙️ Configuration (Memory, Timeout, etc.)

Setting	Range / note
Memory	128 MB – 10,240 MB. CPU scales with memory (more memory = more vCPU).
Timeout	1 s – 900 s (15 min) max.
Ephemeral storage (/tmp)	512 MB – 10,240 MB.
Environment variables	Up to 4 KB total; can be KMS-encrypted.
Architecture	x86_64 or arm64 (Graviton) — arm is cheaper/faster for many workloads.

💡 Tuning **memory** is the main performance lever — more memory also raises CPU, often making functions both faster *and* cheaper. Use **Lambda Power Tuning** to find the sweet spot.

6. ❄️ Cold Starts & Concurrency

- **Cold start** = the latency to set up a new execution environment (download code, start runtime, run init) before the first invocation. Warm reuse avoids it.
- Bigger packages, VPC attachment, and heavy init increase cold starts. Smaller deps + lazy loading reduce them.
- **Concurrency** = number of simultaneous executions.
 - **Reserved concurrency** — caps (and guarantees) a function's max concurrency.
 - **Provisioned concurrency** — pre-warms N environments → **no cold starts** (extra cost).
- **Account concurrency limit** defaults to **1,000** per region (adjustable). Burst scaling adds environments quickly, then linearly.

7. 📦 Deployment Packages & Layers

- **Package types**: a **.zip archive** (code + deps) or a **container image** (up to 10 GB, OCI).
- **Zip limits**: 50 MB zipped (direct upload), 250 MB unzipped; larger via S3.
- **Layers** — share common libraries/dependencies across functions (up to 5 layers per function). Keeps deployment packages small.
- **Runtimes**: managed (Node, Python, Java, Go, .NET, Ruby) or **custom runtime** via the Runtime API / `provided.al2`.

8. 🗑️ Permissions (Execution Role vs Resource Policy)

Two distinct permission directions — don't confuse them:

	Execution Role	Resource-based Policy
Question	What can the function call ?	Who can invoke the function ?
Type	IAM role assumed by Lambda	Policy attached to the function
Example	Read S3, write DynamoDB, put logs	Allow S3/EventBridge/API GW to invoke

The function always needs `logs:CreateLogGroup/Stream/PutLogEvents` (usually via the `AWSLambdaBasicExecutionRole` managed policy) to write to CloudWatch.

9. 🌐 Networking (VPC Access)

- By default a Lambda runs in an AWS-managed network with internet access.

- Attach it to a **VPC** (subnets + security group) to reach private resources (RDS, ElastiCache, internal services).
- A VPC-attached Lambda has **no internet by default** — it needs a **NAT Gateway** (in a private subnet) for outbound internet, or **VPC Endpoints** for AWS services.
- Modern Lambda uses **Hyperplane ENIs** (shared), so VPC cold-start penalty is now minimal.

10. 📦 Versions & Aliases

- **\$LATEST** — the mutable working version.
- **Published version** — an immutable snapshot (`:1` , `:2` ...).
- **Alias** — a named, movable pointer to a version (`prod` , `staging`).
- Aliases enable **weighted/canary deploys** (e.g. 10% traffic to v2, 90% to v1) and clean rollbacks.

11. 🔄 Error Handling & Retries

- **Synchronous** invokes: the error returns to the caller; **no automatic retry** (caller decides).
- **Asynchronous** invokes: Lambda **retries twice** (3 total attempts) over time; failures go to a **Dead-Letter Queue (SQS/SNS)** or an **on-failure destination**.
- **Stream/poll** sources (Kinesis/DynamoDB): retry by **batch** until success or record expiry; can block the shard — use **bisect-on-error**, **max retry**, and a failure destination.
- **Destinations** (async) route success/failure events to SQS, SNS, Lambda, or EventBridge.

12. 📊 Monitoring & Logging

- **CloudWatch Logs** — automatic; `console.log` / `print` lands in a log group per function.
- **CloudWatch Metrics** — Invocations, Errors, Duration, Throttles, ConcurrentExecutions.
- **AWS X-Ray** — distributed tracing of the function and its downstream calls.
- **Lambda Insights** — enhanced per-invocation system metrics.
- **CloudTrail** — API/management audit.

13. 💰 Pricing

You pay for:

1. **Requests** — per invocation (first 1M/month free, then per million).
2. **Duration** — GB-seconds = (memory ÷ 1024) × execution time, billed per **1 ms**.
3. **Extras** — Provisioned Concurrency (per GB-s reserved), ephemeral storage above 512 MB, data transfer.

🔧 **Cost tips:** right-size **memory** (it sets CPU too) · use **arm64/Graviton** · trim dependencies and init time · avoid idle Provisioned Concurrency · batch records from streams/queues · keep functions short and single-purpose.

14. 📖 Common CLI Commands

```
aws lambda create-function --function-name myfn --runtime python3.12 \
  --handler app.handler --role arn:aws:iam::123:role/lambda-role \
  --zip-file fileb://function.zip

aws lambda invoke --function-name myfn --payload '{"key":"val"}' out.json
aws lambda update-function-code --function-name myfn --zip-file fileb://function.zip
aws lambda update-function-configuration --function-name myfn --memory-size 512 --timeout 30
aws lambda publish-version --function-name myfn
aws lambda create-alias --function-name myfn --name prod --function-version 1
aws lambda list-functions
aws lambda get-function --function-name myfn
```

15. 📌 Limits & Best Practices

Item	Value
Max timeout	15 min (900 s)
Memory	128 MB – 10,240 MB
Ephemeral <code>/tmp</code>	512 MB – 10,240 MB
Default concurrency	1,000 / region (adjustable)
Deployment zip	50 MB (zipped) / 250 MB (unzipped)
Container image	up to 10 GB
Invocation payload	6 MB (sync), 256 KB (async)
Layers per function	5

Best practices: keep functions **small & single-purpose** · do heavy setup in **init** and reuse connections · don't store state in the environment (it's ephemeral) · use **environment variables** (encrypted) for config · least-privilege **execution role** · set sensible **timeouts + DLQ** · use **aliases** for safe deploys · prefer **Provisioned Concurrency** only where cold starts hurt.

16. 🧠 Quick Mental Model

- **Lambda** = run code on events, pay per millisecond, zero servers to manage.
- A **trigger** sends an **event** → Lambda runs your **handler** with an **execution role's** permissions.
- Environments are **reused** (warm) — put setup in **init**; first call may be a **cold start**.
- Tune **memory** (it also sets CPU); cap blast radius with **reserved**, kill cold starts with **provisioned** concurrency.
- **Sync** = caller waits, no retry; **async** = queued + retried + DLQ; **streams** = batched polling.
- Attach to a **VPC** for private resources (needs NAT/endpoints for outbound).
- Ship safely with **versions + aliases**; watch it with **CloudWatch + X-Ray**.

17. ? Common Interview Questions

Rapid-fire questions interviewers ask about Lambda:

- **Q: What is a cold start?** — The latency to set up a new execution environment (download code, start runtime, run init) before the first invocation. Provisioned Concurrency eliminates it.
- **Q: Sync vs async vs stream invocation?** — Sync = caller waits, no auto-retry (API GW). Async = queued, retried twice, DLQ on failure (S3/SNS). Stream/poll = batched records (Kinesis/DynamoDB/SQS).
- **Q: Execution role vs resource-based policy?** — Execution role = what the function **can call**. Resource policy = **who can invoke** the function.
- **Q: How do you tune Lambda performance?** — Increase **memory** (CPU scales with it), trim dependencies, reuse connections via the init phase, use arm64/Graviton.
- **Q: Max timeout and memory?** — 15 minutes; 128 MB–10,240 MB.
- **Q: How to reach a private RDS from Lambda?** — Attach the function to the VPC (subnets + SG); add NAT/endpoints if it also needs outbound internet.
- **Q: How do versions and aliases help?** — Immutable versions + movable aliases enable canary/weighted deploys and clean rollbacks.
- **Q: When NOT to use Lambda?** — Long-running (>15 min) jobs, very high steady throughput where always-on compute is cheaper, or workloads needing heavy local state.

 Notes compiled as a quick reference for AWS Lambda.